

Technical Architecture Document

Implementation-ready specification for a Pomodoro productivity application inspired by the functional model of pomodorotimer.online. Revised scope: anonymous, local-first, no login/signup, no user authentication.

1. Architecture Summary

Use a client-first Next.js application. The authoritative runtime state lives in browser state plus durable browser storage. No database or authentication service is required for the initial product.

2. Recommended Stack

Layer	Technology	Purpose
Framework	Next.js + React + TypeScript	Application, routing, SEO pages
Styling	Tailwind CSS	Responsive design system
State	Zustand	Runtime client state
Timer	Web Worker + Date.now() deadline	Accurate countdown
Small persistence	localStorage	Preferences and current timer snapshot
Structured persistence	IndexedDB	Tasks, sessions, stats, challenges, images
Cross-tab	BroadcastChannel	Synchronize commands/state
Offline	Service Worker + Web App Manifest	PWA/offline shell
Audio	Web Audio/HTMLAudioElement	Alarm and ambient audio
Testing	Vitest + RTL + Playwright	Unit/integration/E2E
Deployment	Vercel or static-compatible hosting	Production delivery

3. Runtime Data Flow

```
UI -> Zustand store -> domain services -> localStorage / IndexedDB -> BroadcastChannel -> other tabs
Timer UI <- timer worker ticks <- persisted deadline Service Worker -> cached application
shell/assets
```

4. Timer State Machine

Recommended states: idle, running, paused, completed. Modes: focus, short_break, long_break. Events include START, PAUSE, RESUME, RESET, SKIP, TICK, COMPLETE, CHANGE_MODE, and RESTORE.

Deadline algorithm

```
deadline = Date.now() + remainingMs remainingMs = max(0, deadline - Date.now()) On pause:
remainingMs = max(0, deadline - Date.now()) deadline = null On resume: deadline = Date.now() +
remainingMs
```

5. Persistence Strategy

- Persist timer snapshot immediately on state-changing commands.
- Use IndexedDB transactions for task/session mutations.

- Do not write to storage every second; the deadline makes per-second persistence unnecessary.
- Restore persisted timer state on startup and recompute remaining time.
- Version IndexedDB migrations explicitly.

6. Cross-Tab Coordination

Broadcast timer commands and authoritative snapshots through a named BroadcastChannel. Use a leader/ownership mechanism or last-command timestamp so two tabs cannot independently complete the same session. Completion writes must be idempotent.

7. Offline/PWA

Cache the application shell, essential icons, alarm assets, and selected bundled backgrounds/audio where licensing and size permit. Do not make timer operation depend on network availability. Version caches and clean obsolete caches on service-worker activation.

8. Suggested Source Structure

```
src/ app/ components/{timer,tasks,stats,challenge,music,backgrounds,settings}/
workers/timer.worker.ts stores/{timer,task,settings,stats}.store.ts
services/{timer,storage,sync,audio,challenge}.service.ts db/{indexeddb,migrations,repositories}/
hooks/ types/ constants/ tests/
```

9. Optional Server Boundary

A server may later serve public configuration, analytics endpoints, or licensed asset metadata. It must not become a dependency for core timer operation. Personal task/session storage remains local unless a future cloud-sync feature is intentionally introduced.

10. Performance Targets

- Avoid continuous React re-renders outside timer display/progress components.
- Lazy-load non-critical backgrounds and audio.
- Compress custom images before IndexedDB storage.
- Keep first-load JavaScript and cached assets controlled.
- Use reduced-motion preferences and pause decorative animations when appropriate.